

Combinatorial & Change-Impact Analysis

Slides by Sam Allen

Problem of Analysis

- The ideal way to test which combinations of model constant values result in erroneous output would be to test every single combination of model constant values.
- However, having many model constants that can each take many values will result in too many cases to test.
- Combinatorial testing reduces the number of tests while ensuring pseudo-comprehensive testing.

Combinatorial Analysis Background

- *NIST* research found most software bugs are caused by combinations of a few parameters¹.
- By testing all combinations of a certain number of parameters, most software bugs can be found with much fewer test cases.
- *ACTS* can generate covering arrays from 1-way to 6-way or with mixed coverage².

1. *Automated Combinatorial Testing for Software (ACTS)*. NIST Computer Security Division. (2016, July 20). <https://csrc.nist.gov/groups/SNS/acts/>

2. *User Guide for ACTS*. NIST. (n.d.). https://csrc.nist.gov/csrc/media/Projects/automated-combinatorial-testing-for-software/documents/acts_user_guide_for_basic1.0_and_advanced3.3_versions_nov23.pdf

Our Implementation

- The model constants, which determine how `LakeTemperature` runs, are used as the parameters for the combinatorial test cases.
- Booleans can be either 0 or 1.
- For integers, we linearly interpolate over their range to get up to 11 values.
- For floats, we linearly interpolate over their range to get exactly 11 values.
- *ACTS* generates covering arrays from 2-way to 5-way and outputs a CSV file, from which *NetCDF* files are created using a Python script (`csv2nc`). These are run on the model using the *NetCDF* Cases Sampling Plugin.

Test Oracle Problem

- Scientific software verification is difficult due to lacking a test oracle.
- We implement checks that validate output, supported by the Fault Finding Analysis Plugin.
- Checks include ranges of outputs, signs, enumerated values, finiteness, relationships to other inputs/outputs, and certain physical laws and restraints.

DOI 2 Data

- Degree of Interaction (DOI) 2 results in 133 test cases when only changing the values of constants known to effect the output of LakeTemperature.

33 out of 133 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_combined_heat_content_finite	5	0	95	0
check_combined_heat_content_not_less_than_soil_heat_content	98	0	2	0
check_energy_conservation	15	67	18	0
check_energy_conservation_residual_finite	5	0	95	0
check_errsoi_almost_zero	75	0	25	0
check_freezing_latent_heat	0	100	0	0
check_ground_heat_flux_finite	100	0	0	0
check_ground_methane_conductance_finite	73	0	27	0
check_ground_net_heat_flux_finite	100	0	0	0
check_ground_sensible_heat_flux_finite	100	0	0	0
check_ice_mass_fraction_finite	100	0	0	0
check_ice_thickness_finite	33	0	67	0
check_icethick_col_is_sum	100	0	0	0
check_imelt_uses_valid_enum_values	100	0	0	0
check_lake_layer_ice_fraction_is_fraction	100	0	0	0
check_lake_temperature_finite	5	0	95	0
check_lake_transport_resistance_finite	73	0	27	0
check_lake_water_transport_resistance_not_negative	100	0	0	0
check_melting_latent_heat	0	100	0	0
check_methane_conductance_allowed_without_ice	50	50	0	0
check_methane_conductance_frozen_lake	16	84	0	0
check_methane_conductance_gated_by_ice	16	84	0	0
check_methane_conductance_non_negative	50	50	0	0
check_net_snow_melt_not_negative	100	0	0	0
check_new_snow_melt_rate_finite	100	0	0	0
check_saved_eddy_conductivity_finite	100	0	0	0
check_saved_eddy_conductivity_not_negative	100	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	100	0	0	0
check_snow_depth_finite	100	0	0	0
check_snow_depth_not_negative	100	0	0	0
check_snow_freeze_rate_finite	100	0	0	0
check_snow_freeze_rate_not_negative	100	0	0	0
check_snow_layer_flag_finite	100	0	0	0
check_snow_melt_flux_not_negative	100	0	0	0
check_snow_melt_heat_flux_finite	100	0	0	0
check_snow_melt_heat_flux_not_negative	100	0	0	0
check_snow_melt_water_flux_finite	100	0	0	0
check_snow_water_equivalent_finite	100	0	0	0
check_soil_heat_content_finite	5	0	95	0
check_soil_heat_content_not_negative	93	0	7	0
check_soil_snow_temperature_finite	5	0	95	0
check_surface_absorption_finite	100	0	0	0
check_surface_absorption_fraction_is_fraction	100	0	0	0
check_temp_around_freezing_where_lake_is_almost_frozen	0	100	0	0
check_total_sensible_heat_flux_finite	100	0	0	0
check_water_snow_equivalent_not_negative	100	0	0	0

DOI 3 Data

- DOI 3 results in 3,516 test cases when only changing the values of constants known to effect the output of LakeTemperature.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

DOI 4 Data

- DOI 4 results in 50,031 test cases when only changing the values of constants known to effect the output of LakeTemperature.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

DOI 5 Data

- DOI 5 results in 689,735 test cases when only changing the values of constants known to effect the output of LakeTemperature.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

Results

- Our test cases originally had invalid values for constants, causing crashes.
- Some checks were interpreted incorrectly by us and had to be fixed.
- Tests checking for things other than non-finite values had to ignore non-finite values, otherwise they'd fail and output an inaccurate error message.
- *SPEL* might be initializing data with nan values that LakeTemperature then never overwrites or LakeTemperature might be outputting data with nan values.
- Largely, LakeTemperature output data was valid and within physical bounds. However, edge cases resulted in some physically impossible output, likely due to combinations between a few model constants.

Change-Impact Analysis

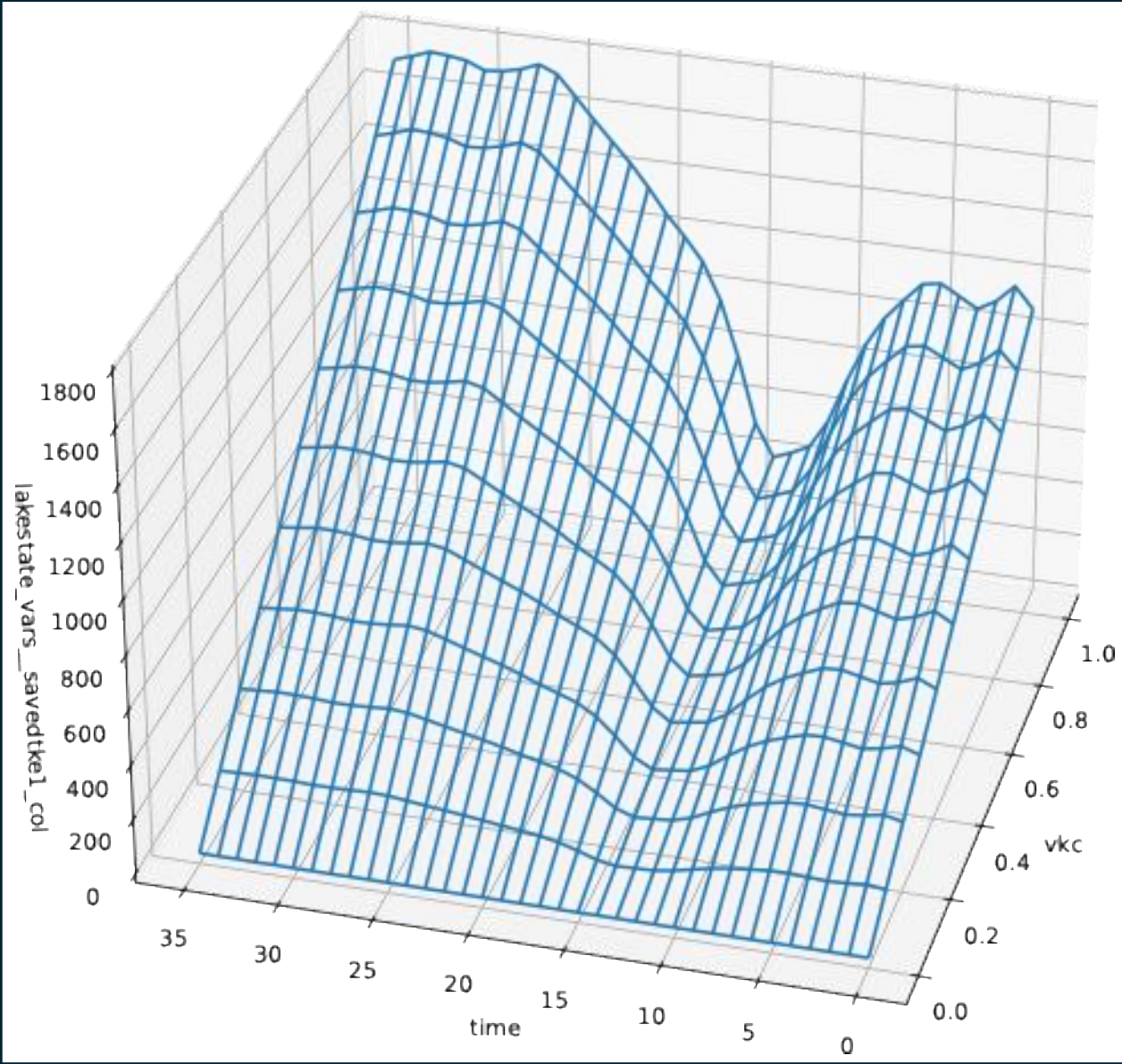
- A method of visualizing the impact of changing a single model constant at a time on the model output.

Challenges

- Most model constants effect multiple model outputs
- Model output is multidimensional
- To solve these problems, we can choose a single column from the output data, use time as the x-axis, use the one model constant we changed as the y-axis, and use a single model output as the z-axis.

Example Graph

- This graph shows how the model constant vk_c affects the model output *lakestate_vars__savedtk_e1_col* over a period of 35 time steps for the lake column numbered 336.



Graphing Process

- Output data in the form of a NetCDF file is used.
- From the output data is pulled the values for the model constant vk_c .
- Time is just interpolated from 0 to 35.
- From the output data is pulled the values for the model output *lakestate_vars__savedtke1_col* at the column numbered 336.
- Using matplotlib, all three dimensions are graphed.

Change Detection Analysis

- Change Detection Analysis seeks to find how specific changes to individual model constants affect model output.
- It does this by changing a single model constant across its range of values while keeping all other model constants the same.

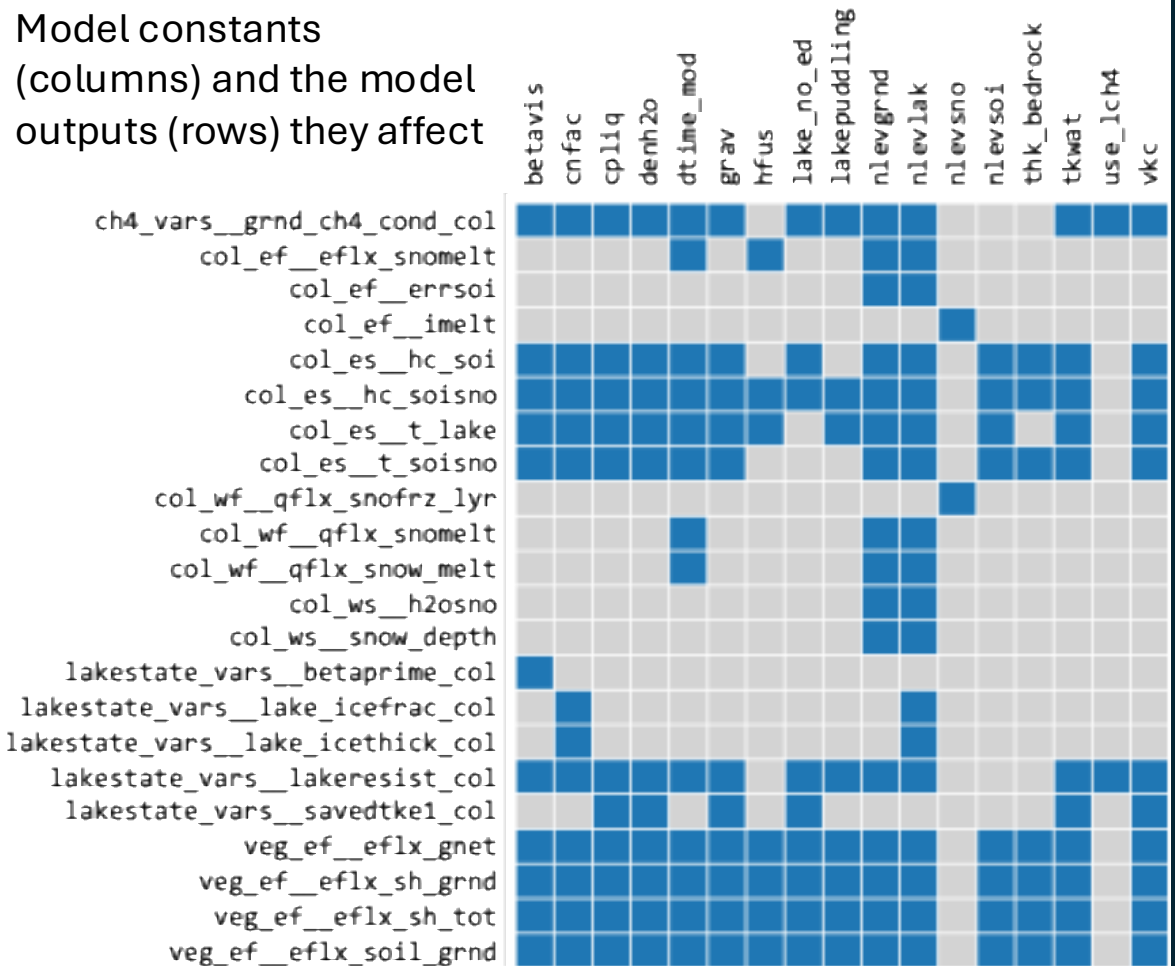
Model constants
that had no effect on
any model output

Preliminary Results

Model output that didn't
change in response to
any model constants

cpice
denice
depthcrit
iulog
mixfact
pudz
tkair
tkice

Model constants
(columns) and the model
outputs (rows) they affect



top_pp_active
lun_pp_topunit
lun_pp_itype
lun_pp_lakpoi
lun_pp_urbpoi
lun_pp_active
lakestate_vars__etal_col
lakestate_vars__lake_raw_col
lakestate_vars__ks_col
lakestate_vars__ws_col
soilstate_vars__watsat_col
soilstate_vars__tkmg_col
soilstate_vars__tkdry_col
soilstate_vars__tksat_col
soilstate_vars__csol_col
solarabs_vars__sabg_patch
solarabs_vars__sabg_lyr_patch
solarabs_vars__fsds_nir_d_patch
solarabs_vars__fsds_nir_i_patch
solarabs_vars__fsr_nir_d_patch
solarabs_vars__fsr_nir_i_patch
col_pp_gridcell
col_pp_topunit
col_pp_landunit
col_pp_itype
col_pp_active
col_pp_snl
col_pp_dz
col_pp_z
col_pp_zi
col_pp_dz_lake
col_pp_z_lake
col_pp_lakedepth
col_es__t_grnd
col_ws__h2osoi_liq
col_ws__h2osoi_ice
col_ws__frac_iceold
col_wf__qflx_snofrz
veg_pp_topunit
veg_pp_landunit
veg_pp_column
veg_pp_itype
veg_pp_active

Assumptions

- However, initial analysis was built on a few key assumptions.
- First, that not all the constants had an effect on the model output.
- Second, that model constants did not influence what values other model constants could take.
- Third, that model constants effects on model output aren't determined by preconditions being met.
- These assumptions were based on the initial results from the change detection analysis plugin.

Rationale

- To attempt to determine which model constants affect which model outputs, 11 cases were created for every model constant where every model constant stayed consistent at a single default value except the one which was being tested, which was linearly interpolated to get 11 values evenly spread across its valid range.
- This assumed that each model constant's values are independent of each other and that no preconditions have to be met for a constant to have an effect on model output.

Usefulness

- By discussing with the domain scientists and examining LakeTemperature source code, these assumptions turned out to be wrong, meaning the results don't include all of model outputs that the model constants effect.
- However, this doesn't discount the results of Change Detection Analysis.
- It led to the discovery that certain constants only have an effect on the output under certain conditions.
- Its results are also still valid and useful when specifically referring to the default values used in the test cases.

Problems/Solutions

- So far, the combinatorial test cases were only using the constants assumed to have an impact on model output. Now, they include all of the constants so that the results of the combinatorial test cases do not depend on previous results being correct or not.
- The Change Detection analysis might be redone in the future by examining preconditions through LakeTemperature source code but future testing, such as combinatorial testing and change-impact analysis, will not depend on these results.